



4



16



1920 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

مقدمة إلى Methods في Java

د. محمد الأحمد / د. عبد الرحمن اللحام



RB Informatics; 30/04/2025

البرمجة 2

في Java ال method أو الدالة هي مجموعة من الأوامر البرمجية التي تؤدي مهمة محددة. يتم تعريف ال method دائماً داخل كلاس.

هيكل method الأساسي:

لتحديد ال method في java، يجب أن تحدد العناصر التالية:

1. Access Modifier (محدد الوصول): مثل public, private، وهو يحدد مستوى رؤية ال Method.
2. Static Keyword (اختياري): كلمة static تشير إلى أن ال Method تابعة لكلاس نفسها وليس لكائن (object) معين من ال Class.
3. Return Type (نوع القيمة المرجعة): نوع البيانات التي سترجعها ال Method عند انتهائها. إذا كانت ال Method لا ترجع أي قيمة، نستخدم void.
4. Method Name (اسم ال Method): الاسم الذي ستستخدمه لاستدعاء ال Method.
5. Parameters (المعاملات أو الوسائط): قائمة بالمتغيرات (parameters) التي يمكن تمريرها إلى ال Method عند استدعائها. توضع بين قوسين ().
6. Method Body (جسم ال Method): الأوامر البرمجية الفعلية التي تنفذها ال Method، وتوضع بين قوسين معقوفين {}.
7. Return Statement (جملة الإرجاع): إذا كان ال Method نوع إرجاع غير void، يجب أن تحتوي على جملة return متبوعة بالقيمة المراد إرجاعها.

الصيغة العامة لتعريف ال method:

```
[access-modifier] [static] return-type method-name(parameters) {
    // java code block (الدالة جسم)
    return value; // (void الإرجاع نوع لم يكن إذا)
}
```

مثال 1 (مثال على Method):

```
public float getAmountDue (String cust) {
    // method variables (الدالة متغيرات)
    int numberOfDays;
    float due;
    float lateFee = 1.50F;
    String customerName;

    // method body (الدالة جسم)
    numberOfDays = getOverDueDays();
    due = numberOfDays * lateFee;
    customerName = getCustomerName(cust);

    return due; // Return statement (الارجاع جملة)
}
```

- public: هو الـ Access Modifier.
- float: هو الـ Return Type (سترجع الـ Method قيمة من نوع float).
- getAmountDue: هو الـ Method Name.
- (String cust): هي الـ Parameters (تستقبل الـ Method معامل واحد اسمه cust من نوع String).
- {} هي الـ Method Body.
- int numberOfDays;, float due ... إلخ: هي متغيرات محلية (method variables) خاصة بهذه الـ Method.
- return due: ترجع قيمة المتغير due.

المتغيرات (Variables) في Java:

- قواعد تعريف المتغيرات:
 - يمكنك تعريف المتغيرات في أي مكان داخل كتلة الـ Class، وخارج أي Method (تعرف هذه بمتغيرات الـ Fields or attributes).
 - داخل الـ Method، يجب تعريف المتغيرات قبل استخدامها.
 - من الشائع تعريف متغيرات الـ Class في بداية كتلة الـ Class.
 - Scope (نطاق) أو Visibility (رؤية) المتغيرات يتم تحديده بواسطة الكتلة البرمجية (code block) التي عرفت فيها. فالمتغير المعرف داخل كتلة معينة يكون مرئيا فقط داخل تلك الكتلة وتوابعها.

تهيئة المتغيرات:

❖ متغيرات الـ Method (Local Variables أو Method Variables): يجب أن تقوم بتهيئتها (إعطائها قيمة أولية) بشكل صريح قبل استخدامها.

❖ متغيرات الـ Class (Class Variables/Static Variables) ومتغيرات الكائن (Instance Variables): يتم تهيئتها تلقائياً بقيم افتراضية إذا لم تقم بتهيئتها صراحة (مثلاً، 0 للأرقام، null للكائنات).

❖ مثال 2 (مثال على متغيرات في سياق Method) :

```
public float getAmountDue (String cust) {
    float due = 0;          // Method variable, تهيئتها تم
    int numberOfDays = 0;   // Method variable, تهيئتها تم
    float lateFee = 1.50F;  // Method variable, تهيئتها تم

    { // جديدة برمجة كتلة بداية (new code block)
        int tempCount = 1; // Temporary variable (مؤقت متغير)
        // الكتلة هذه داخل فقط مرئي المتغير هذا
        due = numberOfDays * lateFee;
        tempCount++;
    } // البرمجة الكتلة نهاية

    // هنا مرئيا ليس tempCount

    return due;
}
```

❖ due, numberOfDays, lateFee هي method variables معرفة في نطاق الـ Method بأكملها.

❖ tempCount هو temporary variable معرف داخل كتلة برمجة فرعية، وبالتالي نطاقه يقتصر على تلك الكتلة فقط.

❖ الـ Class Methods (الـ Static Methods): الـ Class Methods (التي تعرف أيضاً بـ Static Methods) هي دوال ترتبط بالـ Class نفسها بدلا من الارتباط بكائن معين من تلك الـ Class.

خصائص الـ Class Methods:

- Shared by all instances (مشاركة بين جميع كائنات الـ Class): لا يتم إنشاء نسخة منها لكل كائن.
- مفيدة للتعامل مع Class Variables (المتغيرات التي تم تعريفها كـ static).
- يتم تعريفها باستخدام الكلمة المفتاحية static.

```
public static void increaseMinPrice (double inc){
//    أيضاً static variable يجب أن يكون minPrice
    minPrice += inc;
}
```

- طريقة الاستدعاء: يتم استدعاء الـ Class Method باستخدام اسم الـ Class مباشرة، أو من خلال مرجع كائن.

```
Movie.increaseMinPrice;(0.50)
mov1.increaseMinPrice;(0.50)
```

What Are Class Methods? :

- لا تحتوي على this reference: بما أنها لا تعمل على كائن محدد، فلا يوجد لديها مرجع this يشير إلى "الكائن الحالي".
- الوصول المحدود: يمكنها الوصول فقط إلى Class Variables (المتغيرات الـ static) و Class Methods (الـ static الـ Methods) الأخرى في نفس الـ Class.
- يمكنك إنشاء Method لاستخدامها خارج سياق أي كائن.
- الـ static الـ Methods يمكنها فقط استدعاء static الـ Methods أخرى بشكل مباشر.
- الـ static الـ Methods لا يمكنها الإشارة إلى super (الـ Class الأب) أو دوالها.

مثال 3:

```
public class Movie {
    private static float price = 3.50f;
    private String rating;
    public static void setPrice (float newPrice) {
        price = newPrice;
    }
    public String getRating()
        return String;}}
```

دعنا نرى بعض الاستدعاءات وما إذا كانت قانونية:

- `Movie.setPrice(3.98f);` قانوني. نستدعي static method باستخدام اسم الـ Class.
- `Movie mov1 = new Movie(...);`
- `mov1.setPrice(3.98f);` قانوني. الـ Method لا تزال static ويمكنها الوصول فقط للمتغيرات الـ static.
- `String a = Movie.getRating();` غير قانوني. `getRating()` هي instance method (ليست static) وتحتاج إلى كائن لاستدعائها.
- `String b = mov1.getRating();` قانوني. نستدعي instance method على كائن `mov1`.

أمثلة شائعة لـ static methods ومتغيرات في Java:

الـ `public static void main(String[] args)` Method الرئيسية في أي برنامج Java

هي static.

الـ `Math.sqrt()`: دالة حساب الجذر التربيعي في الـ Class `Math` هي static.

الـ `System.out.println()`: `out` هو static field في الـ Class `System`، و `println()` هي

instance method للكائن `out`.

```
public class MyClass {
    public static void main(String[] args) {
        double num, root;
        ...
        root = Math.sqrt(num);
        System.out.println("Root is " + root);
    }
}
```

الـ Method Overloading :

الـ Method Overloading هي ميزة في Java تسمح بتعريف أكثر من Method

بنفس الاسم داخل نفس الـ Class، طالما أن لها مجموعات مختلفة من الـ

Parameters.

- كيف يتم التمييز بين الـ Overloaded Methods؟
- Method Signature : يقوم الـ Compiler بالتمييز بين الـ Overloaded Methods بناءً على بصمتها. تتكون بصمة الـ Method من:

1. اسم الـ Method.

2. عدد ونوع وترتيب الـ Arguments.

- Return Type (نوع القيمة المرجعة): لا يعتبر جزء من بصمة الـ Method. هذا يعني أنه لا يمكنك تعريف دالتين بنفس الاسم ونفس قائمة المعاملات حتى لو اختلف نوع الإرجاع.
- الـ Overloaded Methods لا تحتاج بالضرورة أن يكون لها نفس عدد الـ Parameters.
- حتى الاختلاف في ترتيب الـ Parameters كافي للتمييز بين دالتين.
- * أهمية الـ Method Overloading
- تعد أساسية للسماح باستخدام نفس اسم الـ Method مع أنواع مختلفة من الـ Arguments، مما يجعل الكود أكثر قابلية للقراءة.

الـ overloading هي static polymorphism / compile time polymorphism

أمثلة على الـ Method Overloading :

👉 مثال :

```
public class Movie {
    private float price;

    public void setPrice() { // Method 1
        price = 3.50F;
    }

    public void setPrice(float newPrice) { // Method 2 (Overloaded)
        price = newPrice;
    }
    // ...
}
```

👉 الاستدعاء:

```
class Tree {
    int height;

    Tree() { // Constructor 1
        System.out.println("Planting a seedling");
        height = 0;
    }

    Tree(int i) { // Constructor 2
        System.out.println("Creating new Tree that is " + i + " feet tall");
        height = i;
    }
}
```

```
void info() { // Method 1
    System.out.println("Tree is " + height + " feet tall");
}

void info(String s) { // Method 2 (Overloaded)
    System.out.println(s + ": Tree is " + height + " feet tall");
}

public class BM {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            Tree t = new Tree(i); // يستدعي Constructor 2
            t.info(); // يستدعي info()
            t.info("overloaded method"); // يستدعي info(String)
        }
        // Overloaded constructor:
        new Tree(); // يستدعي Constructor 1
    }
}
```

⚡ الـ Overloading مع الأنواع الأولية (Primitives)

⚡ عندما تمرر قيمة من نوع أولي (primitive type) إلى Method محملة (overloaded method)، يتبع Java قواعد محددة لاختيار الـ Method المناسبة:

- 1- التطابق التام (Exact Match): يبحث أولاً عن Method تتطابق مُعاملاتها تماماً مع نوع القيمة الممررة.
- 2- الترقية (Promotion): إذا لم يجد تطابقاً تاماً، يحاول ترقية النوع الأولي إلى نوع أكبر يمكن أن يستوعبه أحد الـ Methods المحملة. ترتيب الترقية المعتاد هو:
byte -> short -> int -> long -> float -> double
char -> int (ثم يمكن أن يترقى كـ int)

⚡ أمثلة على سلوك الـ Overloading مع Primitives:

⚡ لنفترض أن لدينا الـ Methods التالية:

```
void f1(char x) { System.out.println("f1(char)"); }
void f1(int x) { System.out.println("f1(int)"); }
void f1(long x) { System.out.println("f1(long)"); }
```

```
void f2(int x) { System.out.println("f2(int)"); }
void f2(float x) { System.out.println("f2(float)"); }
```

- f1('a'); سيستدعي f1(char). (تطابق تام)
- f1(5); سيستدعي f1(int). (تطابق تام)
- f1(b); byte b = 10; إذا لم يكن هناك f1(byte) أو f1(short)، سيستدعي f1(int) بعد ترقية byte إلى int.
- f2(10L); (حيث L10 هو long) إذا لم يكن هناك f2(long)، قد يحدث خطأ أو يتم البحث عن ترقية إلى float إذا كانت موجودة، ولكن الأولوية للتطابق الأقرب. في هذه الحالة، إذا لم تكن f2(long) موجودة، سيتم البحث عن f2(float) (لأن long يمكن تحويله إلى float مع فقدان دقة محتمل). إذا كانت f2(double) موجودة وهي الأقرب بعد long، فسيتم استدعاؤها. سيناريوهات الترقية الدقيقة تعتمد على المجموعة الكاملة من الـ overloaded methods.

■ ملاحظة خارجية: من المهم ملاحظة أن الـ Compiler يحاول العثور على "أفضل" تطابق. إذا كان هناك غموض (مثلاً، يمكن ترقية قيمة إلى نوعين مختلفين موجودين كـ overloaded methods بنفس "المسافة" من الترقية)، قد ينتج عن ذلك خطأ في الترجمة (compilation error).

*عدم إمكانية الـ Overloading بناءً على نوع الإرجاع فقط:

لا يمكنك تعريف دالتين لهما نفس الاسم ونفس قائمة المعاملات ولكن تختلفان فقط في نوع القيمة المرجعة.

```
//
{} ()void f
{} ()int f // خطأ: f معرف مسبقاً
```

هذا غير مسموح به وسينتج خطأ في الترجمة
السبب هو أن الـ Compiler لا يمكنه التمييز بينهما عندما يتم استدعاء الـ Method بدون استخدام قيمتها المرجعة،
كما في المثال التالي حيث يتم استدعاء f1() دون تخزين نتيجتها:
f1(); // كيف سيعرف الـ Compiler أي f1() يستدعي إذا كان هناك نسختان بناءً على نوع الإرجاع فقط؟
- الـ this Reference
الكلمة المفتاحية this هي مرجع خاص في Java.

↪ داخل Instance Method أو Constructor: يشير this إلى الكائن الحالي (the

current object) الذي تم استدعاء الـ Method أو الـ Constructor من أجله.

↪ كيف يعمل؟: عندما تستدعي instance method على كائن ما، مثل a.f(1); ، يقوم

الـ Compiler "سراً" بتمرير مرجع للكائن a إلى الـ Method. كأن الاستدعاء تحول

إلى شيء يشبه Banana.f(a, 1); (هذا تمثيل لكيفية عمله داخلياً).

يمكن استخدام this للحصول على مرجع الكائن الحالي داخل الـ Method.

مثال على استخدام this لإرجاع الكائن الحالي:

```
public class Leaf {
    int i = 0;

    Leaf increment() {
        i++;
        return this; // الحالي Leaf الكائن مرجع يرجع
    }

    void print() {
        System.out.println("i = " + i);
    }

    public static void main(String[] args) {
        Leaf x = new Leaf();
        x.increment().increment().increment().print(); // this إرجاع بفضل متسلسلة استدعاءات
        // الناتج: i = 3
    }
}
```

في المثال أعلاه، كل استدعاء لـ increment() يرجع الكائن x نفسه، مما يسمح بتسلسل الاستدعاءات.

متى يجب استخدام this بشكل صريح؟

هناك حالتان رئيسيتان يجب فيهما استخدام this بشكل صريح:

عندما يكون اسم Instance Variable محجوبا (hidden) بواسطة Formal Argument (معامل رسمي) للـ Method يحمل نفس الاسم.

```
public class Movie {
    private String rating; // Instance variable

    public void setRating(String rating) { // 'rating' هو هنا argument
        // نفسها argument الـ قيمة ستعين: rating = rating; // كُتبت إذا
        this.rating = rating; // 'this.rating' إلى يشير instance variable
        // 'rating' إلى يشير (بدون this) argument }}
    }
```

في هذا المثال، `this.rating = newRating` و `rating = newRating` متكافئتان إذا كان اسم الـ `argument` مختلف (مثل `newRating`). ولكن إذا كان اسم الـ `argument` هو نفسه اسم الـ `instance variable` (مثلاً، `rating`)، فإن `this.rating` ضروري للوصول إلى الـ `instance variable`.
 2- عندما تحتاج إلى تمرير مرجع إلى الكائن الحالي كوسيط (`argument`) إلى `Method` أخرى.

```
class A {
    int x = 5;

    void F() {
        if (x == 22) {
            fff(this); // fff إلى الحالي A الكائن تمرير
        }
        System.out.println(x);
    }

    void fff(A S) { // النوع من كائن تستقبل A
        S.x = 8; // الممرر بالكائن الخاص x الحقل على تعدل
    }
}

public class BM {
    public static void main(String[] args) {
        A t = new A(), y = new A();
        t.x = 99; y.x = 22;
        t.F(); // 99 يطبع -> (يتحقق لم الشرط) 99 يبقى t في x
        y.F(); // 8 يطبع -> 8 يساوي y في x فأصبح، fff(y) استدعى، (تحقق الشرط) 22 كان y في x
        System.out.println("-----");
        System.out.println(t.x); // 99 يطبع
        System.out.println(y.x); // 8 يطبع
    }
}
```

في هذا المثال، داخل `y.F()`، الشرط `x == 22` يكون صحيحاً، فيتم استدعاء `fff(this)` حيث `this` هنا يشير إلى الكائن `y`. الـ `Method fff` تقوم بتغيير قيمة `x` للكائن الممرر (الذي هو `y`) إلى 8.

- الكلمة المفتاحية final

الكلمة المفتاحية final في Java تستخدم للإشارة إلى أن شيئاً ما لا يمكن تعديله بعد تهيئته. يمكن تطبيقها على المتغيرات، الـ Methods، والـ Classes.

final Data

مع الأنواع الأولية (Primitives): عندما يعرف متغير أولي (مثل int, float, boolean) ك final، تصبح قيمته ثابتة (constant) ولا يمكن تغييرها بعد إعطائها قيمة أولية.

```
final int MAX_USERS = 100;
// MAX_USERS = 200; // خطأ، لا يمكن تغيير قيمة final variable
```

- مع مراجع الكائنات (Object References): عندما يعرف مرجع كائن ك final، فإن المرجع نفسه يصبح ثابتاً. هذا يعني أنه بمجرد تهيئة المرجع ليشير إلى كائن معين، لا يمكن تغيير هذا المرجع ليشير إلى كائن آخر.
- static final Fields: الحقل الذي يكون static و final في نفس الوقت يمثل قطعة واحدة من الذاكرة لا يمكن تغييرها، أي ثابت على مستوى الـ Class.
- Java لا توفر طريقة لجعل أي كائن عشوائي ثابتاً بالكامل (immutable) بمجرد استخدام final على مرجعه فقط.
- static final Fields: الحقل الذي يكون static و final في نفس الوقت يمثل قطعة واحدة من الذاكرة لا يمكن تغييرها، أي ثابت على مستوى الـ Class.

```
class Configuration {
    public static final String VERSION = "1.0.2";
}
// Configuration.VERSION = "1.1.0"; // خطأ
```

Blank finals

- Java تسمح بإنشاء blank finals، وهي حقول تعلن ك final ولكن لا تعطى قيمة أولية عند الإعلان.
- التهيئة الإلزامية: في جميع الحالات، يجب تهيئة الـ blank final قبل استخدامه، ويضمن الـ Compiler ذلك.

مكان التهيئة:

- بالنسبة لـ instance blank finals (ليست static): يجب تهيئتها في كل constructor من الـ Class.
- المرونة: توفر blank finals مرونة أكبر، حيث يمكن أن يكون للحقل الـ final داخل الـ Class قيمة مختلفة لكل كائن (في حالة instance blank finals)، ومع ذلك يحتفظ بخاصية عدم القابلية للتغيير بعد التهيئة الأولية.

مثال على BlankFinal

```
class Poppet {
    private int i;
    Poppet(int ii) { i = ii; }
}

public class BlankFinal {
    private final int i = 0; // Initialized final (مهيأ نهائي)
    private final int j; // Blank final (instance)
    private final Poppet p; // Blank final reference (فارغ نهائي مرجع)

    // Constructor: Blank finals الـ في الـ تهئية يجب:
    public BlankFinal() {
        j = 1; // blank final تهئية
        p = new Poppet(1); // blank final reference تهئية
    }

    public BlankFinal(int x) {
        j = x; // blank final تهئية
        p = new Poppet(x); // blank final reference تهئية
    }

    public static void main(String[] args) {
        new BlankFinal(); // Poppet(1) - يشير p, 1 سيكون j
        new BlankFinal(47); // Poppet(47) - يشير p, 47 سيكون j
    }
}
```

final Parameters:

يمكنك جعل معاملات الـ Method نهائية (final) بتعريفها كذلك في قائمة المعاملات.

- التأثير: هذا يعني أنه داخل الـ Method، لا يمكنك تغيير ما يشير إليه مرجع المعامل (إذا كان كائناً) أو تغيير قيمة المعامل (إذا كان نوعاً أولياً).

🔗 مثال على final Parameters:

```
class Gizmo {
    public void spin() {}
}

public class FinalParameters {
    void with(final Gizmo g) {
        // g = new Gizmo(); // قانوني غير
        if (g != null) {
            g.spin(); // المرجع نغير لا مسموح،
        }
    }

    void without(Gizmo g) {
        g = new Gizmo(); // قانوني
        g.spin();
    }

    // void f(final int i) {
    //     i++; // قانوني غير
    // }

    int g(final int i) {
        return i + 1; // قانوني
    }

    public static void main(String[] args) {
        FinalArguments bf = new FinalArguments();
        bf.without(null);
        bf.with(null);
    }
}
```

❖ داخل with(final Gizmo g)، لا يمكنك إعادة تعيين و ليشير إلى Gizmo جديد.

❖ داخل f(final int i) (لو كانت موجودة)، لا يمكنك تغيير قيمة i. يمكنك فقط قراءتها.

:final Methods

هناك سببان رئيسيان لجعل الـ Method نهائية (final):

- 1- منع الـ Overriding (إعادة التعريف): السبب الأول هو "قفل" الـ Method لمنع أي Class يرث منها من تغيير سلوكها أو معناها. يستخدم هذا لأسباب تصميمية عندما تريد التأكد من أن سلوك الـ Method يبقى كما هو خلال عملية الوراثة.
- 2- الكفاءة (Efficiency).

:private و final

- أي private methods في الـ Class هي ضمنياً final.
- بما أنك لا تستطيع الوصول إلى private method من خارج الـ Class، فلا يمكنك أصلاً عمل override لها في subclass.
- يمكنك إضافة محدد final إلى private method، لكنه لا يضيف أي معنى إضافي.
- final Classes
- * عندما تعلن أن الـ Class بأكملها final (بوضع الكلمة المفتاحية final قبل تعريف الـ Class)، فإنك تقرر أنك لا تريد أن يتم الوراثة من هذه الـ Class، أو أن تسمح لأي شخص آخر بذلك.

```
final class MyFinalClass {
    // ...
}

// class AnotherClass extends MyFinalClass {} // خطأ، لا يمكن الوراثة من final class
```

↪ **حقول الـ final Class:** حقول الـ final class يمكن أن تكون final أو لا، حسب اختيارك. القواعد نفسها تنطبق على final للحقول بغض النظر عما إذا كانت الـ Class نفسها final.

↪ **Methods في final Class:** بما أن الـ final class تمنع الوراثة، فإن جميع الـ Methods في final class هي ضمنياً final، لأنه لا توجد طريقة لعمل override لها. يمكنك إضافة محدد final إلى Method في final class، لكنه لا يضيف أي معنى.

ل Class String في Java هي final.

استعادة الذاكرة (Reclaiming Memory) و ال Garbage Collection

Java تدير الذاكرة تلقائياً باستخدام عملية تعرف بـ Garbage Collection .

عملية الـ Garbage Collection

- عندما تفقد جميع المراجع (references) إلى كائن ما (أي لم يعد هناك أي متغير يشير إليه)، يصبح هذا الكائن مؤهلاً لـ Garbage Collection.
- يقوم الـ Garbage Collector باستعادة الذاكرة التي كان يستخدمها هذا الكائن.
- عملية الـ Garbage Collection تلقائية (automatic).
- لا يحتاج المبرمج عادة للقيام بأي شيء حيالها، ولكن يمكنه إعطاء "تلميح" للنظام لمحاولة تشغيلها باستخدام `System.gc();`
- استخدام الـ `finalize()` Method
- إذا كان الكائن يحتفظ بموارد خارجية (مثل ملف مفتوح، اتصال بقاعدة بيانات، إلخ)، فقد تحتاج إلى طريقة لتنظيف هذه الموارد قبل أن يتم تدمير الكائن.
- يمكنك توفير Method خاصة اسمها `finalize()` في تلك الـ Class.

```
Public class Movie {
...
    public void finalize() {
        System.out.println("Goodbye");
    }
}
```

- ولكن لا يوجد ضمان بشأن متى سيتم استدعاء `finalize()`، أو حتى ما إذا كان سيتم استدعاؤها قبل خروج البرنامج.

```
class Book {
    boolean checkedOut = false;
    Book (boolean checkOut) { checkedOut = checkOut; }
    void checkIn () { checkedOut = false; }
    public void finalize() {
        if (checkedOut)
            System.out.println("Error: checked out");
    }
}

public class BM {
    public static void main(String[] args) {
        Book novel = new Book(true);
    }
}
```

```
novel.checkIn();
// Drop the reference, forget to checkIn();
new Book(true);
// Force garbage collection & finalization:
System.gc();
}}
```

1. الفئة Book:

- تحتوي على خاصية checkedOut التي تحدد ما إذا كان الكتاب مستعاراً أم لا.
- تحتوي على منشئ (constructor) يأخذ قيمة بوليانية (boolean) لتعيين حالة الاستعارة.
- تحتوي على دالة checkIn لإرجاع الكتاب.
- تحتوي على دالة finalize التي تستدعى قبل حذف الكائن من الذاكرة. إذا كان الكتاب لا يزال مستعاراً عند استدعاء هذه الدالة، سيتم طباعة رسالة خطأ.

2. الفئة BM:

- في الدالة main، يتم إنشاء كائن novel من الفئة Book مع تعيين checkedOut إلى true.
- بعد ذلك، يتم استدعاء checkIn لإرجاع الكتاب، مما يجعل checkedOut تساوي false.
- يتم إنشاء كائن جديد من Book بدون الاحتفاظ بالإشارة إليه، مما يعني أنه لن يمكن الوصول إليه لاحقاً.
- يتم استدعاء Garbage Collector باستخدام System.gc()، والذي يحاول تحرير الذاكرة المستخدمة بواسطة الكائنات غير المستخدمة.

